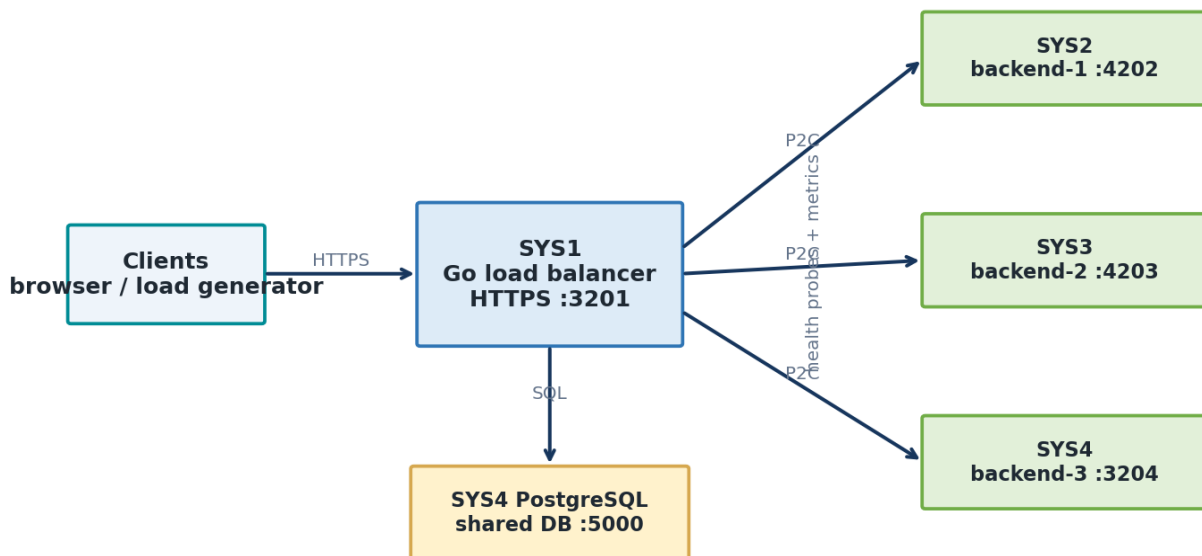


CHATFAT MESSAGING

Lab 6 project report

Secure persistent group chat with performance-based dynamic load balancing

Deployment topology



Student	Aditya K Yadav	Roll number	12340100
Assigned host	10.1.75.51	Submission endpoint	https://10.1.75.51:3201
Systems	Sys1, Sys2, Sys3, Sys4	Report date	September 15, 2026

Submission note: the public service is exposed only through the HTTPS load balancer. The certificate is self-signed for the lab environment, so a browser must accept the certificate warning once.

1. Executive summary

This project extends a secure WebSocket group-chat application into a four-system deployment. Sys1 runs a Go reverse proxy and HTTPS entry point. Sys2, Sys3 and Sys4 run identical Node.js chat backends. All three backends use the same PostgreSQL database on Sys4, so a message written through one backend is available through the others.

The main contribution is performance-based routing. The balancer uses Power of Two Choices rather than fixed round-robin. It compares two healthy candidates using latency, event-loop lag and in-flight work, then sends the request to the lower-scoring candidate. Health probes remove failed backends from selection and restore them after recovery. Watchdog processes restart failed services.

Measured outcome: the persistent threshold sweep completed 3,864 to 3,916 requests per run with zero dropout. The recovery check completed 600 of 600 requests successfully at 1,642.65 requests per second, with near-even distribution across all three backends.

1.1 Requirements traceability

Assignment requirement	Implementation and evidence
Three backend systems	Sys2, Sys3 and Sys4 run the same Node.js application.
Client-facing load balancer	Sys1 exposes HTTPS on port 3201; backend addresses remain internal.
Performance-based balancing	P2C selection uses service latency EWMA, event-loop lag and in-flight work.
Unhealthy backend handling	Health probes, three-failure eviction, one-success restoration and watchdog restart.
Persistent shared data	All backends use PostgreSQL on Sys4 through the same DATABASE_URL.
No duplicate messages	Unique message IDs and database uniqueness constraints make retries idempotent.
Required APIs	/message accepts client-name and msg; /feed returns persisted messages.
Variable load generator	Go tools support users, concurrency, message sizes, intervals and utilization sampling.

2. Architecture and request pipeline

The architecture separates the public entry point from the application nodes. This is important for two reasons. First, clients use one stable URL even when a backend is removed. Second, routing decisions can use backend health and performance without exposing internal addresses.

Request and message pipeline



Figure 1. A normal API request or WebSocket upgrade follows the same entry path. The database commit occurs before a successful persistent message response is reported.

Port and responsibility map

Service	Address	Host	Purpose
Public client endpoint	10.1.75.51:3201	Sys1	Only endpoint submitted to evaluator
Load balancer to backend 1	172.17.0.3:4202	Sys2	HTTP proxy and health target
Load balancer to backend 2	172.17.0.4:4203	Sys3	HTTP proxy and health target
Load balancer to backend 3	172.17.0.5:3204	Sys4	HTTP proxy and health target
Shared database	172.17.0.5:5000	Sys4	PostgreSQL for all backends

Figure 2. Port map and responsibility map. The only address submitted to the evaluator is the public HTTPS endpoint.

2.1 Technologies and responsibilities

Layer	Technology	Reason for selection
Client	HTML, JavaScript, WebCrypto	Browser-native encryption and signing with no extra client runtime.
Realtime	Node.js, raw WebSocket	Efficient event-driven chat transport and room events.
Authentication	Better Auth	Session-based signup, login and sign-out with trusted-origin checks.
Persistence	PostgreSQL and pg	Shared durable store and database-level uniqueness guarantees.
Balancer	Go net/http	Small concurrent proxy with explicit health and routing control.
Recovery	Shell watchdogs	Automatic restart using the existing system tools.

3. Dynamic load-balancing strategy

Round-robin assumes that all backends have equal capacity and equal current load. That assumption is not reliable when one Node process is handling a queue, a host is busy, or a backend is recovering. P2C gives most of the benefit of load-aware scheduling without polling every candidate for every request.

For each request, the balancer samples two healthy backends. Each backend exposes measurements that contribute to a score: recent service latency is smoothed with an EWMA, event-loop lag indicates whether the Node process is responsive, and in-flight count represents queue pressure. The candidate with the lower score receives the request. The process repeats as measurements change, so traffic moves away from a slow backend without permanently starving it.

3.1 Deployed controls

Control	Value	Reasoning
Selection	P2C	Adapts to live performance instead of assuming equal backends.
Load threshold	200 ms	Responsive operational policy for switching away from rising latency.
Health interval	250 ms	Detects failure quickly without probing on every request.
Health timeout	600 ms	Bounds the cost of a health probe.
Unhealthy threshold	3 consecutive failures	Prevents a single delayed probe from causing eviction.
Healthy threshold	1 successful probe	Restores a recovered backend quickly.
Backend timeout	45 seconds	Prevents an indefinitely blocked proxy request.

Backend health state machine



Timeouts under overload return 504 and remain a health-loop decision. This prevents false eviction of a busy but alive backend.

Figure 3. Health state machine. The separate failure threshold is hysteresis: it distinguishes a dead backend from a temporary queue caused by a burst.

3.2 Failure behavior

A failed health probe increments the failure counter. After three consecutive failures the backend is marked unhealthy and removed from routing. A later successful probe restores it. Connection-level proxy errors can also trigger eviction. An overload timeout returns a bounded error but is not treated as permanent death immediately; otherwise a busy backend could be evicted, leave no capacity, and then appear healthy only because the load disappeared.

4. Persistence, consistency and security

All three Node backends point to the same PostgreSQL instance on Sys4. This removes process-local memory as the source of truth and allows any healthy backend to serve the next feed request. The database connection is configured through DATABASE_URL and the same MASTER_KEY is required to read encrypted data consistently.

4.1 Duplicate prevention

Message submission can be retried after a timeout or reconnect. Each message therefore carries a unique message ID. The repository writes the message with a database uniqueness constraint. If the same ID arrives again, the constraint prevents a second row. This is stronger than checking only in application memory because the database remains authoritative when multiple backends receive the same retry concurrently.

Route	Input or output	Persistence behavior
/message	client-name and msg	Creates or safely reuses the message identified by its unique ID.
/feed	Ordered message list	Reads the shared durable feed from PostgreSQL.
/healthz	Health and load snapshot	Used by the balancer and watchdog.

4.2 Security design

Control	Purpose
HTTPS at Sys1	Provides the secure browser context required by WebCrypto and protects the public hop.
Better Auth sessions	Handles account creation, login, session lookup and logout.
AES-GCM room encryption	Encrypts room messages in the browser before transport.
PBKDF2 room-key derivation	Derives room keys from room credentials without sending the raw key.
ECDSA P-256 signatures	Allows clients to verify message authorship and tamper detection.
MASTER_KEY	Protects encrypted data at rest and must be shared by all persistent backends.

Deployment distinction: DATABASE_URL=none is used only for isolated benchmark experiments. The submitted deployment uses the shared PostgreSQL URL. Running one backend without that URL would create inconsistent feeds and would not satisfy the persistence requirement.

5. Experimental method and results

The load tests were run from the repository load-generator tools. A warm-up phase fills connection pools before measurements. The generator records successful and failed requests, response-time percentiles, throughput, backend distribution and load-balancer status. Chat-shaped tests mix POST message operations and feed reads, with variable user counts, message lengths and intervals.

5.1 Threshold sweep using PostgreSQL

Threshold (ms)	Requests	p50 (ms)	p95 (ms)	p99 (ms)	Throughput	Dropout
500	3864	11.96	409.39	460.56	1884.55/s	0.00%
1000	3876	10.63	655.76	829.95	1519.61/s	0.00%
2000	3916	15.13	933.70	1329.42	1130.63/s	0.00%
4000	3892	13.45	1682.55	2129.48	827.96/s	0.00%

Persistent PostgreSQL threshold sweep

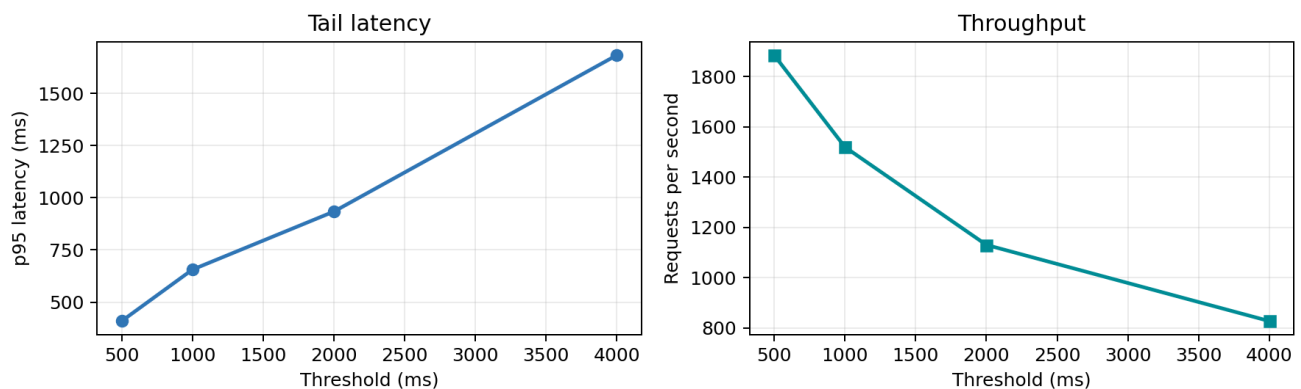


Figure 4. Latency and throughput are plotted separately so their different units do not hide the trend. The 500 ms candidate had the best measured p95 and throughput in this persistent sweep, with zero dropout. The deployed 200 ms value is a more conservative operational setting that moves traffic away from a rising queue earlier. For a fixed workload, the sweep provides evidence for selecting 500 ms; for interactive chat, the 200 ms policy prioritizes responsiveness. This distinction makes the threshold decision explicit rather than claiming that one value is universally optimal.

5.2 Recovery and distribution

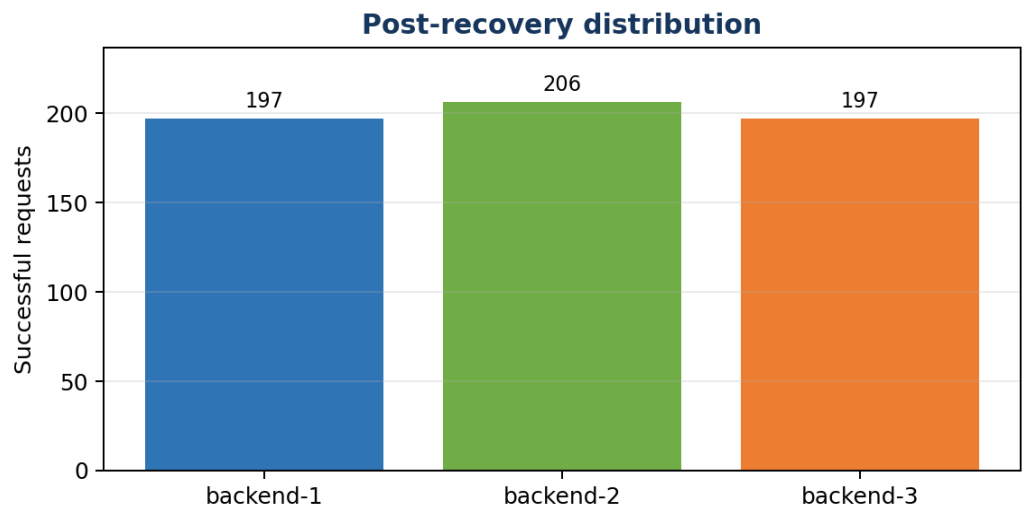


Figure 5. After backend recovery, 600 of 600 requests succeeded. Distribution was 197/206/197 across the three backends.

Metric	Observed result
Requests	600
Successful / failed	600 / 0
Throughput	1642.65 requests/s
p50 / p95	14.62 ms / 36.34 ms
HTTP status	600 responses with status 200

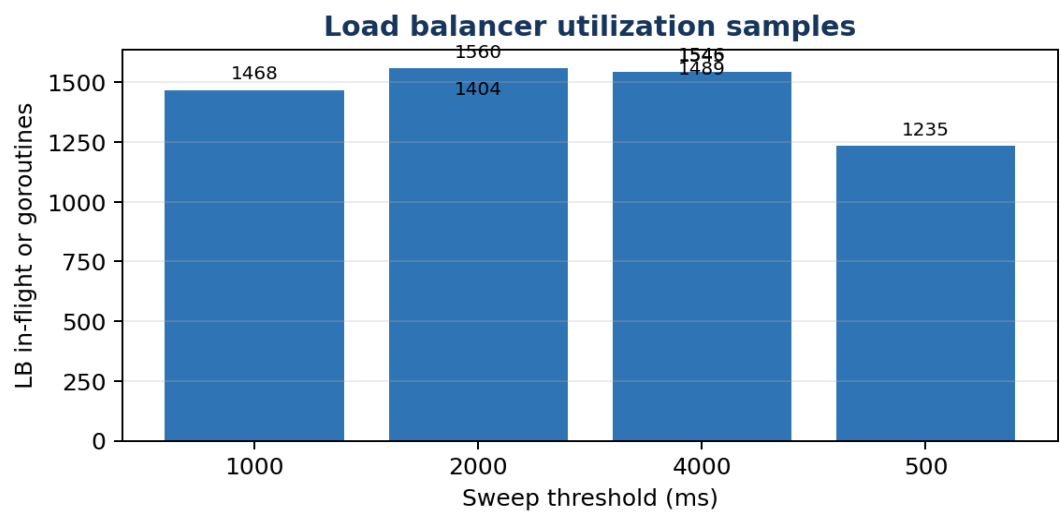


Figure 6. Checked-in load-balancer utilization samples. The capture records in-flight or goroutine counts and backend scores. Host CPU fields were not populated in this artifact, so no host CPU percentage is inferred from it.

6. Operations, troubleshooting and verification

The deployment was verified at the public endpoint and through backend health checks. The following checks are useful during a demonstration or evaluation:

Check	Command	What it proves
Public health	<code>curl -k https://10.1.75.51:3201/healthz</code>	TLS listener and proxy are reachable.
Backend status	<code>curl -k https://10.1.75.51:3201/lb/status</code>	Alive state, in-flight work and per-backend counts.
LB metrics	<code>curl -k https://10.1.75.51:3201/lb/metrics</code>	Throughput, errors and latency percentiles.
Live LB log	<code>tail -f /tmp/lb.log</code>	Selection and proxy errors in real time.
Backend log	<code>tail -f /tmp/backend-1.log</code>	Application startup and request failures.
Port ownership	<code>ss -ltnp grep ':3204'</code>	Finds a stale process without requiring lsof.

Common faults and fixes

Symptom	Diagnosis and corrective action
HTTP request sent to HTTPS server	Use <code>https://10.1.75.51:3201</code> . The TLS listener received an HTTP request; this is a protocol mismatch.
Browser reports no WebCrypto	Use the HTTPS endpoint and accept the self-signed certificate in the browser.
Better Auth ERR_REQUIRE_ESM	Load the ESM-only Better Auth packages with dynamic import and provide Node 18 WebCrypto.
Backend marked DOWN with EOF	Check whether the backend is plain HTTP while the health probe is also configured for HTTP.
Address already in use	Inspect the exact listener with <code>ss</code> , stop the stale PID, and ensure only one watchdog starts the service.
Database connection fails	Verify port 5000, credentials, listen addresses and the same DATABASE_URL on Sys2, Sys3 and Sys4.

7. Strengths, limitations and future improvements

Strengths

The implementation preserves the secure chat functionality while adding a real deployment layer. It uses one public endpoint, adaptive routing, explicit health state, shared durable storage, database-enforced idempotency, encrypted rooms, session authentication, metrics and automatic recovery. The experiments are reproducible because the load generators and result files are kept with the project.

Limitations

The lab certificate is self-signed, which is appropriate for the assigned private network but not for a public service. PostgreSQL on Sys4 is a shared dependency and therefore a single database failure can affect all backends. The checked-in utilization capture is centered on the load balancer rather than a synchronized CPU trace from all four hosts.

Future improvements

Area	Improvement
Database availability	Use PostgreSQL replication, automated failover and scheduled backups.
Observability	Export synchronized CPU, memory, event-loop lag and request metrics to a central dashboard.
Transport security	Replace the lab certificate with a trusted certificate and automate renewal.
Scaling	Add backends dynamically and use a shared service registry instead of static addresses.
Threshold control	Tune the threshold from a workload-specific objective such as p95 latency under a dropout limit.
Overall conclusion: the system meets the assignment requirements without removing the original secure chat behavior. The evidence demonstrates persistence, adaptive routing, health-aware failure handling and recovery through the single submission endpoint.	